



機器學習_學習筆記系列(24)：決策樹分類(Decision Tree Classifier)



劉智皓 (Chih-Hao Liu) · Follow

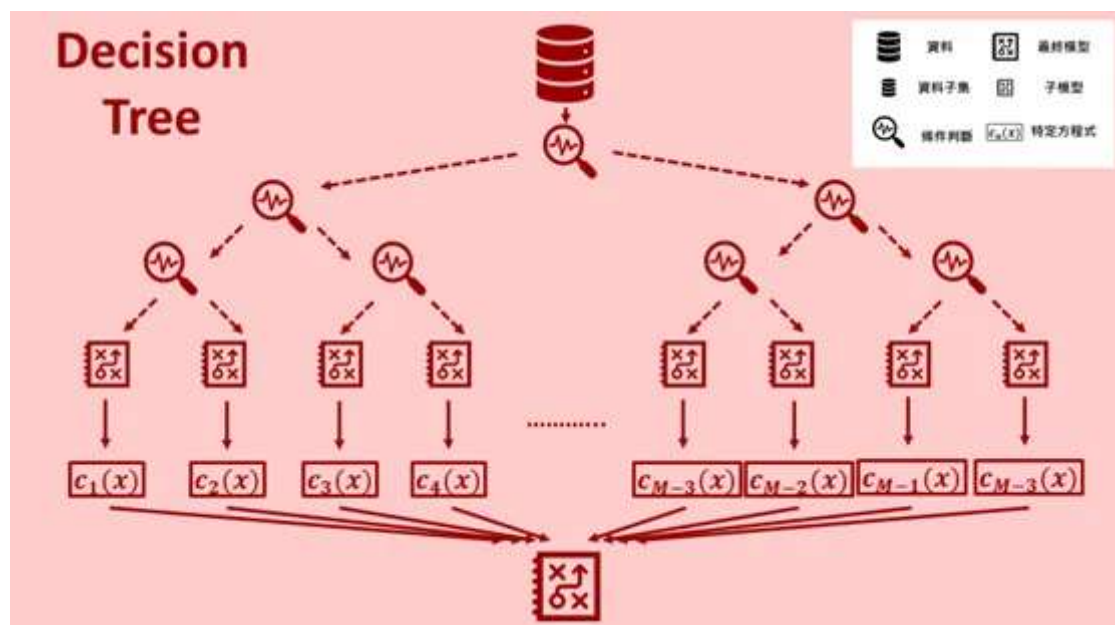
Mar 22, 2021



Share

上一回，我們介紹了各種aggregation models，那我們今天就要來細講之中每個模型，而第一個要講的就是Decision Tree。

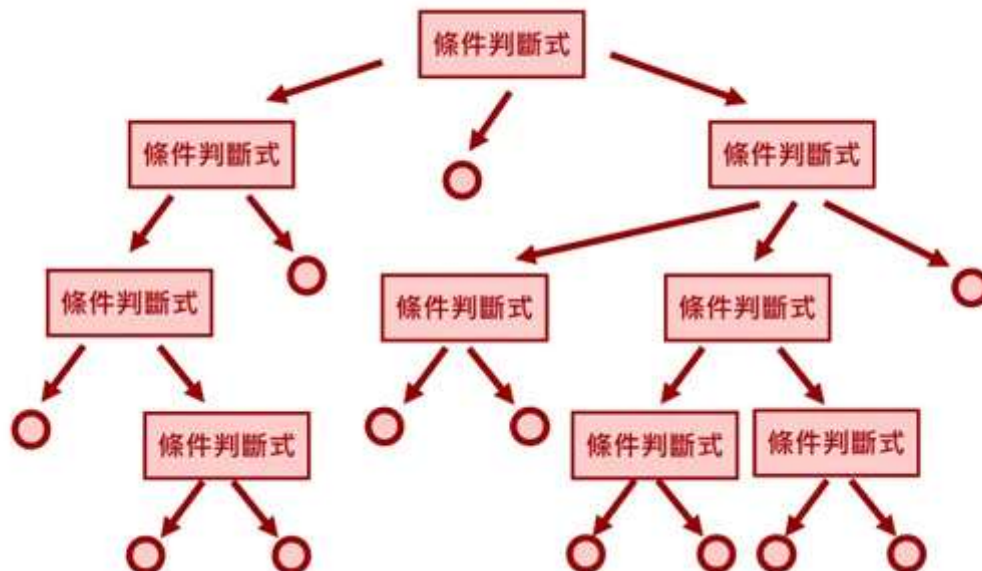
Decision Tree在上一次我們也提到過，他是一種**機器學習演算法**，可以用來分類也可以用來做回歸分析。而decision tree在這方面的專有名詞叫做**Classification and Regression Tree (CART)**。把專有名詞縮寫還能成為一個單字才酷吧！



以decision tree來說我們也提到，當輸入一筆資料後，我們根據條件判斷式做分類，像是上次的例子，分辨貓跟狗，第一層可能看他的體型，第二層可能看他的壽命，然後將這些資料的特徵，按照不同條件判斷式，分類到不能再分類為止，也因為這樣一層一層分支的結構，這個演算法才會叫做decision tree。

Decision Tree Concept

對於decision tree，從圖片裡可以看到，輸入資料後他會依照每一層不同的條件判斷式將資料做分類，直到沒辦法再分下去為止，就產會產生一個子模型，也就是圖中圓圈的部分。

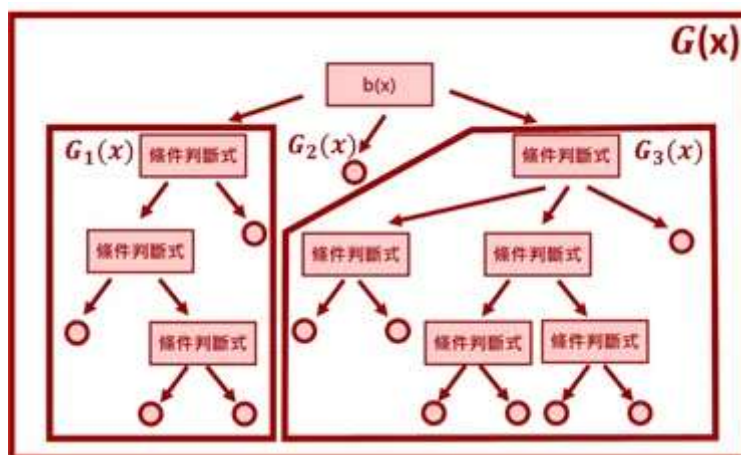


另外我們知道decision tree是一種aggregation models，所以說他一樣可以表示成

$$G(x) = \sum_{c=1}^C \mathbb{I}[b(x) = c] G_c(x)$$

其中C代表整個樹節點的總數，也就是條件判斷式總共有幾個。而b(x)代表的就是節點的條件判斷方程式，G_c代表的就是第c個節點以下的子樹(sub-tree)方程式。

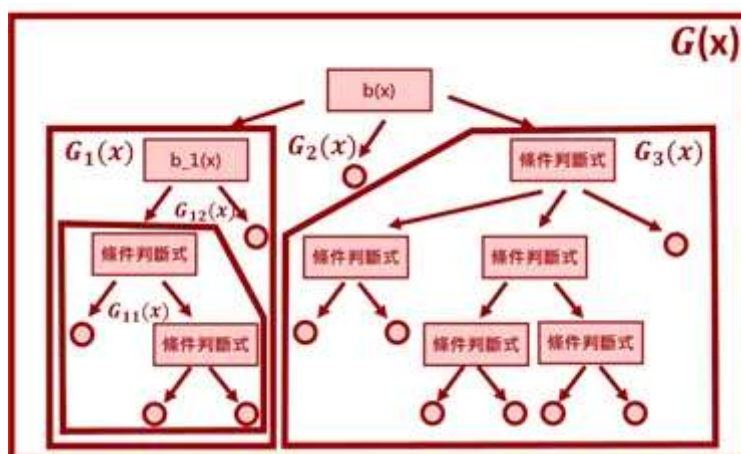
具體來說，以下面那張圖為例，G(x)代表整個tree的方程式，而第一層總共有三個節點，所以C=3，而b(x)就是最上層的條件判斷方程式，所以G_1(x)代表就是左邊的sub-tree。G_2(x)代表就是中間的subtree，而這裡G_2已經到結尾了。而G_3(x)代表右邊的subtree。



而對於 $G_1(x)$ 和 $G_3(x)$ 我們可以繼續用相同的方法分下去，像是

$$G_1(x) = \sum_{c=1}^C \mathbb{I}[b_1(x) = c] G_{1c}(x)$$

其中 b_1 代表 G_1 subtree最上層的條件判斷方程式， G_{1c} 代表下一層中的subtree。而我們可以看到 $G_1(x)$ 底下有兩個subtree，其中一個是左邊的 G_{11} ，而另一個是右邊已經分到盡頭形成葉子的 G_{12} 。



所以我們可以根據這樣遞迴的邏輯將我們整個tree分解到只剩下葉子為止。

Decision Tree Algorithms

對於Decision Tree的演算法[1]，其如下

```

function Decision_Tree( $\text{data } D = \{x_n, y_n\}_{n=1}^N$ )
  if(termination criteria met)
    return hypothesis  $G_t(x)$ 
  else
    learn branch criteria  $b(x)$ 
    split  $D$  to  $C$  parts  $D_c = \{x_n, y_n : b(x) = c\}$ 
    build subtree  $G_c(x) = \text{Decision\_tree}(D_c)$ 

    return  $G(x) = \sum_{c=1}^C \mathbb{I}[b(x) = c] G_c(x)$ 

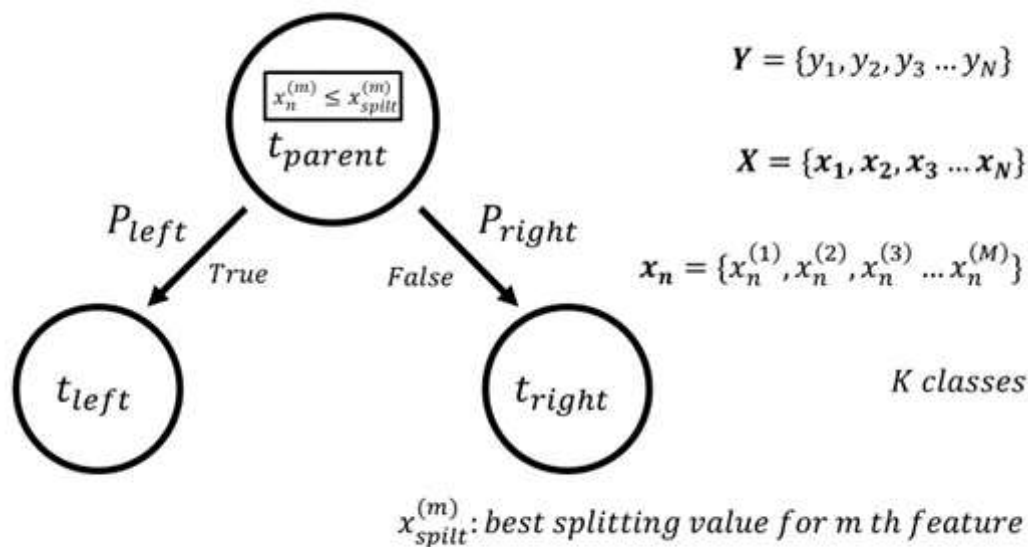
```

這個演算法其實就和資料結構中樹的演算法很像，利用**遞迴呼叫**建立我們的資料。所以我們命名我們的呼叫方程式為Decision_Tree，如果我們所建立的subtree不能再分割了，則表示我們到達葉子，所以在此建立模型 G_t 。若還可以繼續拆，則先建立我們的條件判斷式 $b(x)$ ，然後根據條件判斷式，把我們的資料分成 C 份，接著把這些分好資料再次套入Decision_Tree方程式。

Decision Tree Classifier

那我們現在知道了演算法的建立，接下來就是進入解決分類問題。對於Classification & Regression Tree (CART)來說，我們會把 C 設為2，也就是每次**遇到條件判斷式，就只分成兩個subtree**。

而我們知道對於機器學習的核心概念，就是**最小化預測值和實際值的差異**。所以對於分類問題，我們希望每次經過一個條件判斷，分成兩個subtree的時候，同一類別被分到同一個subtree的比例越高越好。這裡我們畫一個簡單的小圖[2]



設我們的資料有N筆、M個特徵、K個種類。P_left和P_right分別代表被分到左邊和右邊的機率。而每次分支時，我們都會針對一個feature，設定一個值x_split(m)，讓同一類分到同一個subtree的比例最高(錯誤率最小)，所以說x_split(m) ≥ x_n(m)就是我們上面提到的條件判斷方程式b(x)。而要把上述這些東西量化，我們可以把它寫成

$$\min_{x_{split}^{(m)}} P_{left} GI(t_{left}) + P_{right} GI(t_{right})$$

其中GI叫做*Gini index*，其為

$$GI(t) = 1 - \sum_{k=1}^K p(k|t)^2$$

整個式子所代表的就是在每一次分支時，求出最好的x_split(m)，讓兩個subtree的不純度最小化。

而整個Decision Tree演算法的停止機制就是

1. 當我們impurity function等於0(分類乾淨，沒有錯誤點)
2. 剩下的資料label一模一樣沒辦法區分。

Decision Stump

Ok那到這邊，我們一定會想，我們知道演算法了，那麼到底該怎麼分，我們的分類線要長怎樣。所以對於這個部分我們叫要用Decision Stump來分。

Decision Stump做的事情，其實和Decision Tree一樣，針對一個特徵，找到一個值，然後讓GI達到最小，只是decision stump只有parent node和左右兩個child node。所以對於分類方程式為

$$h(x_n^{(m)}) = s \cdot \text{sign}(x_n^{(m)} - x_{\text{split}}^{(m)})$$

其中s為正號或負號，也就是我們的分類線，其垂直於我們所選的feature的向量方向。而實際算出 $x_{\text{split}}(m)$ 的演算法(這裡是我自己想的，所以有更好的演算法拜託告訴我QQ)

第一步：對N筆資料的每個feature做排序，所以會有M個排序好大小的feature sets

第二步：針對每個排好的feature set，從第一筆開始，以現在這筆資料(含)以前當作left subtree，以後當作right subtree計算impurity值

第三部：取出impurity值最低的那筆資料，最後再比較哪個feature sets的impurity最低，把那筆資料和下一筆資料的數值平均就可得到 $x_{\text{split}}(m)$ ，並回傳其所屬的feature

如此一來我們就可以在每一次分支時得到我們的分類線

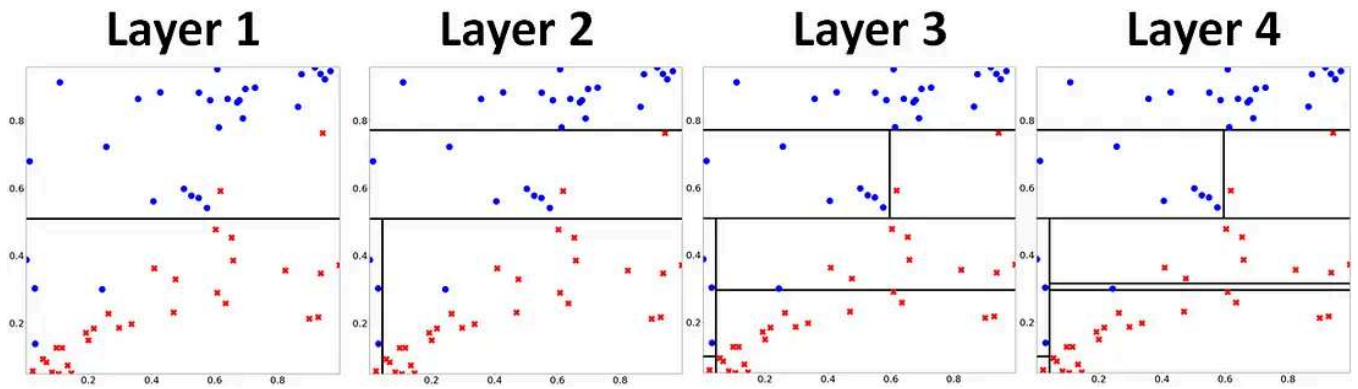
層數限制

對於用Decision Tree演算法都會碰到一個問題，那就是對於N筆資料，如果我們讓演算法直接硬train下去，就會發現他可能會產生N-1個分支。顯然這種情況就是終極的overfitting，你會看到training error=0，但是testing error直接爆炸。所以說我們勢必要對Decision Tree限制其分支的深度或分支的數量。

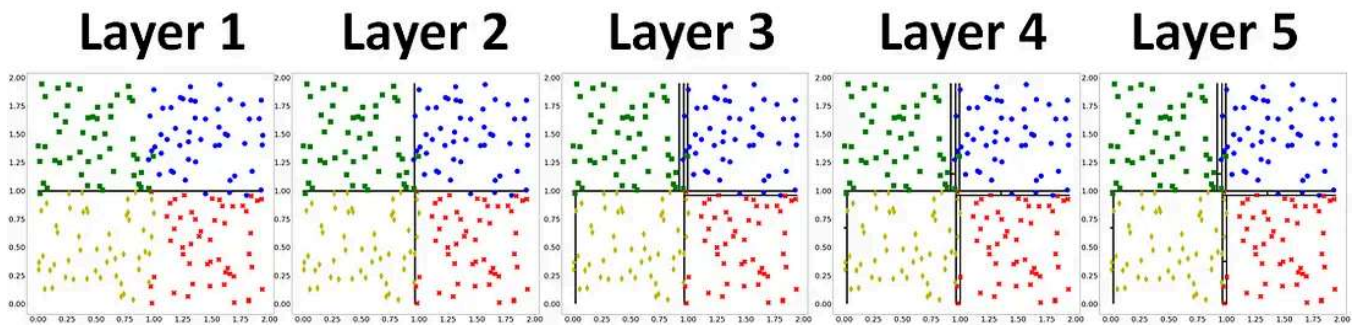
Apply Algorithm

有了上述的方法後，我們用python實現我們的演算法

二元分類



多元分類



我們可以看到，Decision Tree層數越多，分類線就越複雜，出現overfitting的情況，所以在運用Decision Tree做分類的時候，必須去衡量，層數的深度。

Python Sample Code:

Loading

決策樹分類(Decision Tree Classifier)

先引入我們需要的packages

```
In [1]: import os
import numpy as np
import random
import matplotlib.pyplot as plt
```

Plot Decision Line

```
In [2]: def plot_line(x_split,m,x_lim):
# m: 代表第m個feature
# x_split: 分割線
# xlim: 畫分割線的邊界
# xlim[0,0],xlim[0,1]-->分類區橫軸最小值和最大值
# xlim[1,0],xlim[1,1]-->分類區縱軸最小值和最大值
if(m==0):
plt.plot([x_split,x_split],[x_lim[1,0],x_lim[1,1]],'k',linewidth
else:
plt.plot([x_lim[0,0],x_lim[0,1]],[x_split,x_split],'k',linewidth
```

決策樹分類(Decision Tree Classifier).ipynb hosted with ❤ by GitHub

[view raw](#)

Github:

tomohiroliu22/Machine-Learning-Algorithm

Contribute to tomohiroliu22/Machine-Learning-Algorithm development by creating an account on GitHub.

github.com

Reference:

[1] Coursera, 機器學習技法, 林軒田 教授, 國立臺灣大學

[2] Timofeev, R. (2004). Classification and regression trees (CART) theory and applications. *Humboldt University, Berlin*, 1–40.

機器學習

學習筆記

決策樹

Decision Tree

[Follow](#)

Written by 劉智皓 (Chih-Hao Liu)

1K Followers · 4 Following

豬屎屋AI RD，熱愛AI研究、LLM/SD模型、RAG應用、CUDA/HIP加速運算、訓練推論加速，同時也是5G技術愛好者，研讀過3GPP/ETSI/O-RAN Spec和O-RAN/ONAP/OAI開源軟體。

No responses yet



Write a response

What are your thoughts?

More from 劉智皓 (Chih-Hao Liu)